

LineSense AI

A factory operations decision-support system for apparel manufacturing:
four bounded agents over a deterministic domain core

IT 3041 group assignment

*Author names and student numbers to be completed by the team before submission
(see `docs/assessment/contribution-log.md`)*

20 September 2026

Read this before any number in this report

This report describes a system that was really built and really tested, but *not* everything in it has been executed. Three things in particular have never run in the environment that produced this repository, and are marked as such wherever they appear:

- **No live language-model call has ever been made.** No provider API key exists here. Every agent run in this project used the deterministic `fixture` provider, labelled “*Test fixture — not a live AI model*” in the UI, in the API and in every evaluation artefact.
- **No Docker image was ever built or started.** The Compose, Caddy and Keycloak artefacts were validated *statically* only.
- **The full evaluation, the load test, and the full test suites were not run locally**, under this project’s machine heat and workload policy. Each is listed as **PENDING (needs user approval or CI)** with the exact command that would produce it.

Browser end-to-end testing was **dropped by the user** as a scope decision, so there is no Playwright suite, no `make test-e2e` target and **no screenshots** anywhere in this submission.

Repository: `feat/linesense-build`. The running, dated log of what was built and what was actually run is `docs/IMPLEMENTATION_STATUS.md`; the requirement-by-requirement verdict is `docs/assessment/completion-matrix.md`.

Contents

1	Introduction and problem	4
1.1	The problem	4
1.2	What LineSense AI is	4
1.3	Template gap and the honesty rule this report follows	4
2	Domain, users and roles	4
2.1	Demonstration domain	4
2.2	Roles	5
3	Requirements	5
4	System design	5
4.1	Architectural style	5
4.2	Boundaries that are actually enforced	10
4.3	Data model	10
4.4	State design	10
5	Technology stack and exact versions	12
6	The four agents and bounded autonomy	14
6.1	The shape every agent has	14
6.2	What each agent may and may not do	14
6.3	Degradation is explicit, never a silent fallback	14
6.4	The agent task protocol	14
6.5	The orchestration graph	14
6.6	Deterministic engineering calculations	18
6.7	Independent reference fixtures	18
7	Methodology	18
7.1	Deterministic core plus bounded agents	18
7.2	Retrieval	18
7.3	NLP	19
7.4	Evaluation design	19
8	Implementation	21
8.1	What exists, in numbers	21
8.2	Repository layout	21
8.3	How it was built	21
8.4	Frontend	21
9	Workflows	22
9.1	Analysis: planner to canonical report	22
9.2	Approval and application	22
9.3	Quality hold, release and shipment eligibility	24
9.4	Import	24
10	Security	24
10.1	Authentication, sessions and tenancy	24
10.2	Threat model	24
10.3	Prompt injection: what the adversarial test actually proves	24
10.4	Scans (recorded runs)	25
10.5	Accepted residual risks	25
11	Responsible AI implementation	26
12	Evaluation results	26
12.1	Functional verification that <i>was</i> executed	27
12.2	Backup and restore (a real, recorded exercise)	28

13 Reliability, performance and deployment	28
13.1 Reliability	28
13.2 Efficiency	28
13.3 Performance: not measured	29
13.4 Deployment	29
14 Commercialization	31
15 Limitations, and what has not been verified	32
15.1 Never executed in this environment	32
15.2 Known functional limitations	32
15.3 Future work, in the order it should be done	33
16 Conclusion	33
References	33
A Role permission matrix	34
B API surface	35
C Command reference	36
D Architecture decision records	37

1 Introduction and problem

1.1 The problem

An apparel factory answers the same question dozens of times a day, and answers it badly: *can this order finish on time, what is blocking it, what evidence supports that, and what should we do next?* The information needed to answer it is real and already exists — order books, bills of material, stock ledgers, line capacity calendars, cycle-time observations, inspection records, standard operating procedures — but it sits in different spreadsheets, a partially-used ERP, a storekeeper’s notebook and an industrial engineer’s clipboard. Reconciling it is manual, slow, and done under time pressure by the person least able to spare the time.

The consequence is not usually a wrong decision. It is a *late* decision: a material shortage discovered on the day the line was supposed to start, a bottleneck operation nobody quantified, a quality hold that was not visible to the person promising a shipping date.

1.2 What LineSense AI is

LineSense AI is a decision-support application for that question. Four bounded AI agents — **planning/allocation**, **raw materials (RM)**, **industrial engineering (IE)/cycle time** and **quality** — investigate one order, each using scoped read-only tools over a deterministic domain core, and produce evidence-linked findings. Anything they propose is a *recommendation*, never a change: a second human being must approve it and then apply it through a transaction that re-checks every input version before it writes anything.

Three design commitments run through the whole system and are the reason it can be trusted at all:

1. **A language model never establishes business truth.** Stock, capacity, cycle times, quality eligibility and lifecycle transitions are computed by pure Python in `app/domain/` — `Decimal` in, `Decimal` out, no I/O. A model may explain a result, pick one of the options the deterministic code already produced, and cite evidence that already exists. It cannot produce a number, a state, or a verdict.
2. **Every consequential action is a proposal until a human approves it.** The proposer may not approve their own proposal; an approval does not apply anything; and an application that finds any input has moved fails closed.
3. **Nothing is shown without its source.** Every value on screen is labelled “*Calculated from records*”, “*AI recommendation*”, “*Test fixture — not a live AI model*”, “*AI disabled — deterministic results only*”, “*Pending human approval*” or “*Approved by [role/user] at [time]*”.

The system recommends. It does not operate machinery, place purchase orders, or authorise shipments.

1.3 Template gap and the honesty rule this report follows

The official report template was not supplied to this project. This document is self-structured; when the template becomes available, restructure this content into it rather than rewriting the content.

Every quantitative claim below cites the artefact that produced it — a recorded command in `docs/IMPLEMENTATION_STATUS.md`, a test name, a generated document, or a file in the repository. Anything that was not executed is named as not executed. Section 15 is a single list of everything in this category, and the completion matrix (`docs/assessment/completion-matrix.md`) repeats the verdict per requirement.

2 Domain, users and roles

2.1 Demonstration domain

The demonstration organization is *Demo Apparel Group* with two factories, KTN (Katunayake Plant) and BYG (Biyagama Plant). Two factories exist specifically so that access boundaries can be tested: a planner at BYG must not be able to see a KTN order, and a negative test asserts they receive 404, not 403 — the existence of an out-of-scope resource is never revealed.

All data is synthetic and generated deterministically (`app/seed/generator.py`, seed 20260917). Every customer name is obviously fictional (*Customer 07 Apparel Co.*); every worker identifier is a pseudonymous alias code (KTN-OP-014), never a person’s name.

Table 1: The seeded demonstration dataset ([docs/evaluation/synthetic-data.md](#)).

Organizations / factories	1 / 2	Capacity slots (9 lines $\times$ 30 d $\times$ 2 shifts)	
Users / memberships / role assignments	10	Orders (80 KTN, 20 BYG, 1 demo)	
Customers	26	Allocations	
Styles / style operations	12 / $\approx$ 88	Material lots / movements / balances	40 / $\approx$ 600
Materials	20	Reservations / expected receipts	$\approx$ 75 /
BOM versions / lines	16 / $\approx$ 61	Operator aliases / skill records	150 /
Production lines (KTN L1–L6, BYG B1–B3)	9	Cycle observations / line measurements	720 /
SOP corpus documents	30	Inspections / defects / holds / releases	$\approx$ 40 / $\approx$ 32 / 1 /

2.2 Roles

Roles are **additive and explicit**. Nothing is implied: an `org_admin` administers membership and configuration and has *no* business authority — it cannot approve a recommendation, transition an order, or write inventory, IE or quality records. The authoritative matrix is generated from the code (`scripts/gen_role_matrix.py` reads `app.auth.policy.PERMISSIONS`) and is reproduced in Appendix A.

Table 2: The seven roles and what each may do beyond the shared read set.

Role	Authority beyond reading
viewer	none — authorized read-only dashboards and evidence
planner	create/import orders, transition orders, <i>start an analysis run</i> , add notes
storekeeper	material receipts, lot acceptance, issues, corrections, reservations and releases
ie_engineer	record cycle observations, mark an outlier, upload documents
quality_manager	record inspections, place holds, record releases, upload documents
supervisor	everything a planner may do, plus <i>decide</i> and <i>apply</i> recommendations, cancel/dispatch orders, read the audit log, upload documents
org_admin	membership and role administration, audit log, policy list, effective run limits — and no business authority at all

Two supervisors and two quality managers are seeded deliberately: separation of duties requires a second person. The proposer of an analysis can never approve or apply the resulting recommendation, and a quality manager can never release an order against an inspection they recorded themselves.

3 Requirements

`docs/requirements.md` is the requirements baseline: 21 numbered functional requirements (REQ-01...REQ-21) and 5 non-functional requirements (NFR-01...NFR-05), each with acceptance criteria and a “where implemented” pointer. Every one of them is carried into `docs/assessment/completion-matrix.md` with a verdict of *Met*, *Partially met*, *Not verified* or *Not met* and a link to the evidence.

4 System design

4.1 Architectural style

LineSense is a **modular monolith with a separate worker process** (ADR-0001). The four agents are independently specified software components inside one backend codebase; none of them is a deployed microservice. The reasons are recorded in the ADR: at this scale, per-agent services would add deployment, failure and consistency modes without adding capability, and the protocol boundary that matters — a versioned message contract with tenant scoping, budgets and evidence rules — can be enforced just as strictly inside one process boundary.

One PostgreSQL 16 cluster with the `pgvector` extension is the only datastore (ADR-0002): domain data, the job queue, run records, the audit log and the chunk embeddings all live in it. There is no separate vector database, no Kafka, no Redis and no Kubernetes.

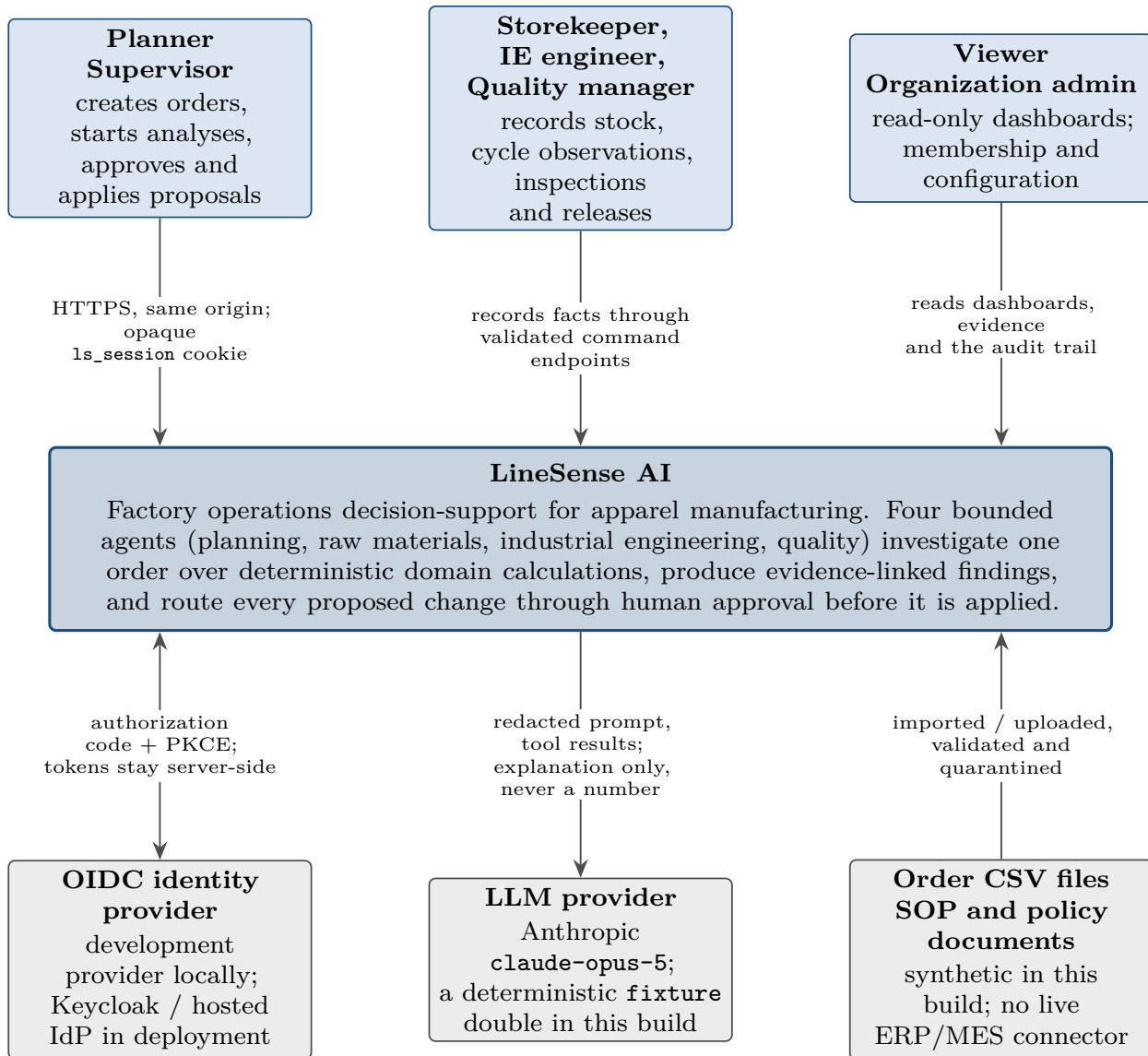


Figure 1: C4 level 1 — system context. The three actor groups, the system, and the three external systems it depends on. The LLM provider is drawn because the boundary exists and is implemented; in this build it is always the deterministic **fixture** double.

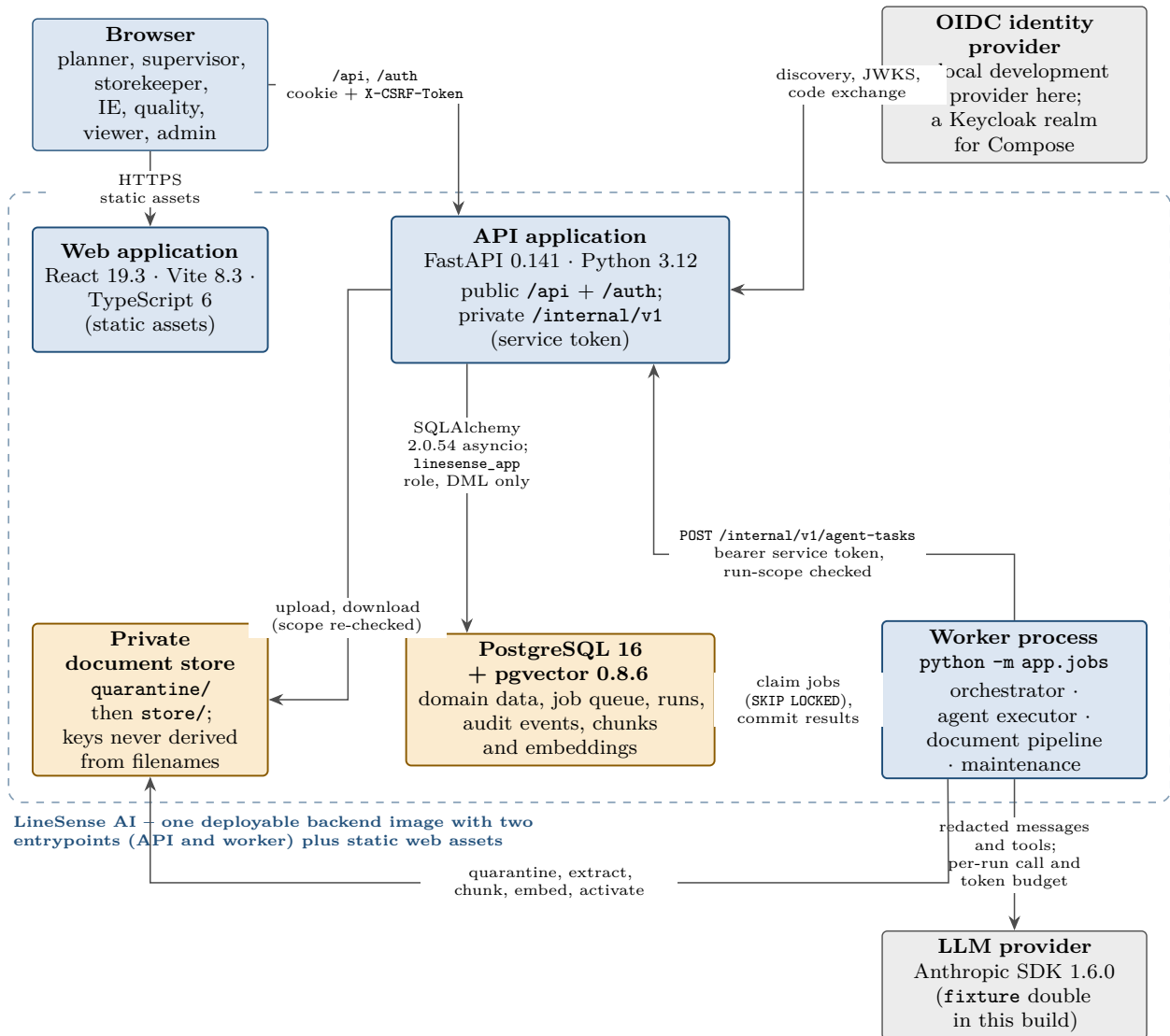


Figure 2: C4 level 2 — containers. API and the internal dispatch endpoint are the *same* application with a public and a private routing boundary; the worker is the same deployable image with a different entrypoint. The browser can reach neither the database, nor the provider key, nor /internal/v1.

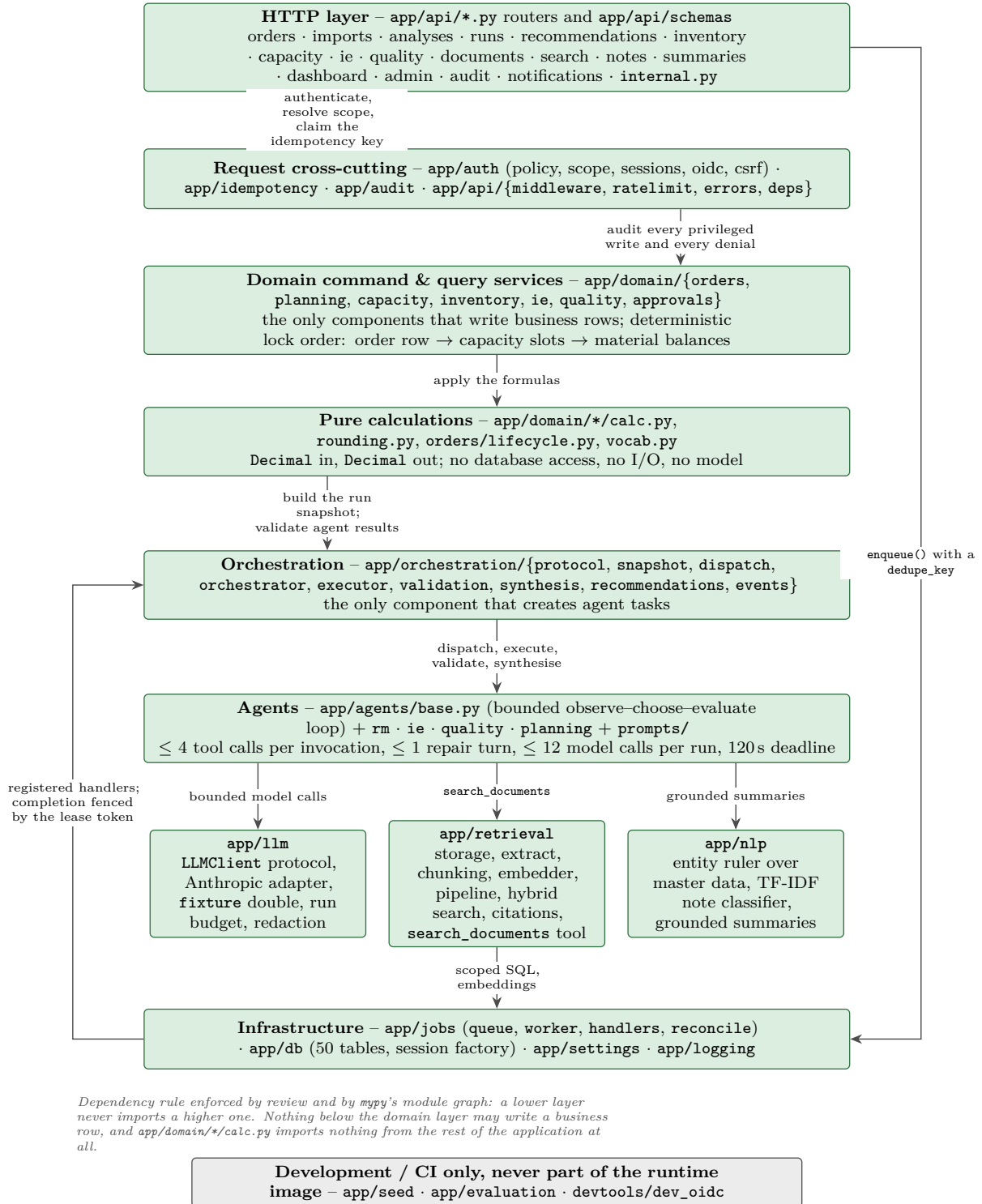


Figure 3: Component view of the backend, collapsed to its dependency layers. The rule a lower layer never imports a higher one is what keeps `app/domain/*/calc.py` free of every other concern: those modules import nothing from the rest of the application at all.

Table 3: Assignment requirement → implementation → evidence.

Brief requirement	What was built	Evidence
Two or more interacting intelligent agents	Four domain agents exchanging typed tasks and results through an orchestrated, versioned HTTP/JSON protocol; planning consumes RM and IE results and is re-dispatched when RM contradicts it	tests/integration/test_four_agents.py; test_two_agent_flow.py; figure 6.5
LLM use	One LLMClient boundary; the model selects among offered tools, writes the explanation and picks one of the deterministic candidate actions	app/llm/ docs/architecture/llm-boundary.m Live provider never exercised
NLP	Entity extraction over authorized master data, note classification, grounded summaries	app/nlp/; micro-F1 0.991 / macro-F1 0.835 on the held-out split (smoke run, §12)
Information retrieval	Lexical tsvector + pgvector cosine, fused by reciprocal-rank fusion, filtered by organization/factory/ACL/version before content reaches a model	app/retrieval/; figure 7; §12
Security	OIDC + PKCE, opaque server sessions, CSRF double-check, scoped authorization on every path, validated uploads, rate limits, audit	tests/security/ (7 modules), docs/security/threat-model.md
Defined agent protocol	schema_version "1.0" envelope/result over POST /internal/v1/agent-tasks with a service token	contracts/agent-task-envelope.s contracts/agent-result.schema.js
Responsible AI	Evidence-linked findings, abstention, pseudonymous worker data, human review of every consequential action	docs/assessment/responsible-ai.m model-card.md, data-card.md
Commercialization	Target market, three tiers, cost model, deployment options, pilot plan	docs/assessment/commercialization §14

4.2 Boundaries that are actually enforced

- The browser holds an **opaque session identifier** and nothing else — no profile data, no provider tokens, no bearer token in `localStorage`. Provider tokens stay server-side after the OIDC code exchange.
- `/internal/v1` is excluded from the public OpenAPI document (`include_in_schema=False`), requires a constant-time-compared bearer service token, and is returned as 404 at the edge by the reverse proxy as an independent control.
- Agents receive **read/compute tools over an immutable run snapshot**. No agent tool accepts SQL, a shell command or a URL; no tool can create another tool; and the tool set of each agent is fixed in code, never derived from request input.
- Only the domain command services write business rows, and they do so under a single documented lock order: order row → recommendation row → capacity slots (ascending id) → material balances (ascending id).

4.3 Data model

The schema is 50 tables in one Alembic revision (`migrations/versions/0001_initial_schema.py`), grouped as identity, demand, capacity, inventory, IE, quality, documents, workflow, decisions and operations. Every table uses UUID keys, explicit units, `Decimal` quantities, UTC timestamps and a `version` column on mutable business records.

Tenant safety is structural, not conventional. Tenant-owned rows carry `organization_id`; plant-specific rows also carry `factory_id`; and every table with both **not null** uses a composite foreign key (`organization_id, factory_id`) → `factories(organization_id, id)`, so a cross-organization factory reference is rejected by the database itself. A dedicated integration test (`test_schema.py::test_cross_org_factory_reference_rejected`) asserts this.

Three further schema-level guarantees worth naming:

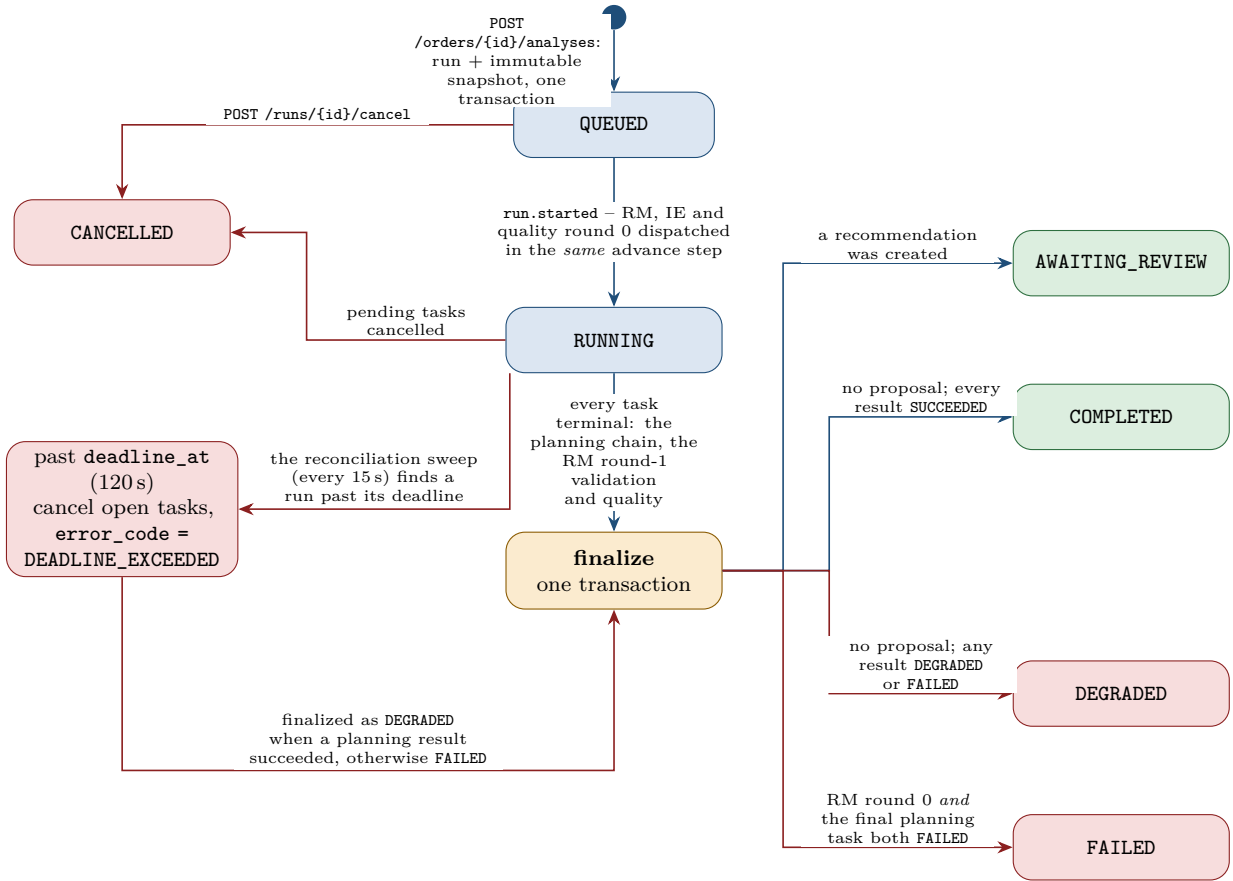
- `audit_events` has **no foreign keys at all** and the runtime role `linesense_app` is granted `SELECT`, `INSERT` on it with `UPDATE`, `DELETE` revoked — append-only at the privilege level, tested by `test_app_role_cannot_update_or_delete_audit_events`.
- Inventory is a **ledger**. Receipts, issues and corrections are `stock_movements` rows; a correction references the movement it corrects; and a lockable `material_balances` row with its own `version` is the thing a transaction takes for update.
- `chunks.tsv` is a persisted generated `TSVECTOR` column with a GIN index, and `chunks.embedding` is a `pgvector.sqlalchemy.Vector(384)` column — lexical and vector search read the same row.

4.4 State design

Four separate state vocabularies are kept separate on purpose, because merging them is exactly how a single misleading “order status” gets invented:

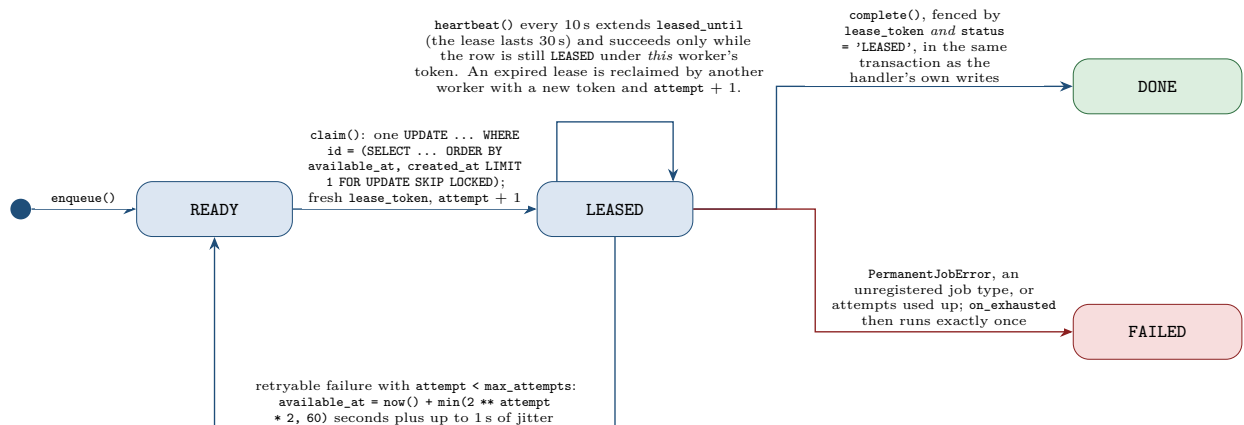
- **Production:** `DRAFT` → `VALIDATED` → `PLANNED` → `IN_PRODUCTION` → `PRODUCTION_COMPLETE` → `DISPATCHED`, with explicit permitted cancellation transitions. Every transition is defined in one central policy table (`app/domain/orders/lifecycle.py`) with actor permissions, preconditions and audit requirements. **There is no `PATCH status` endpoint.**
- **Materials:** `UNKNOWN` | `READY` | `AT_RISK` | `SHORTAGE`.
- **Quality:** `NOT_INSPECTED` | `PENDING` | `HOLD` | `RELEASED`.
- **Analysis (per run):** figure 4.

`shipment_ready` is *derived*, never stored as independent truth and never selected by a model: production complete **and** the required current inspections satisfied **and** no active hold **and** packing/quantity records complete **and** an authorized quality release recorded. Zero inspections yields *unknown*, never *pass*. A test (`test_a_model_all_clear_never_makes_the_shipment_eligible`) drives a scripted model that says “All clear — ready to ship” and asserts that neither the findings, the metric, nor `report.shipment.eligible` change.



Every `orchestrator.advance` delivery locks the run row, does exactly one thing (start, dispatch or finalize), commits, and only then makes a network call – the run lock is never held across the wire. `AWAITING_REVIEW` is the only state that hands work to a human; the approval workflow takes over from there. `POST /api/v1/runs/{id}/retry` is accepted on any terminal state and starts a new run from a fresh snapshot – it never resurrects this row. Cancelling also supersedes the run's open recommendations with `superseded_reason = "RUN_CANCELLED"`. A worker that dies mid-task is recovered either by its job lease expiring or by the same reconciliation sweep, which re-enqueues `orchestrator.advance` for any run quiet for more than 30 s with no runnable job of its own.

Figure 4: The analysis-run state machine. The orchestrator is the only component that creates agent tasks, and every delivery of `orchestrator.advance` does exactly one thing before committing.



Delivery is at least once and is documented as such – never exactly-once. A handler can finish work outside the database and then lose its lease before committing. Fencing guarantees that at most one attempt's database writes commit; handlers must make any external effect idempotent, and `enqueue()`'s `dedupe_key` guarantees a repeated request creates at most one row. `CANCELLED` exists in the schema and is set by no code path. A handler that raises `LeaseLostError` leaves the row to whichever worker now owns it – the job is never marked failed by the worker that lost it.

Figure 5: The durable job and lease state machine. Delivery is at least once and is documented as such; fencing by `lease_token` guarantees that at most one attempt's database writes commit.

5 Technology stack and exact versions

Versions below are the **locked** versions from `services/backend/uv.lock` and `apps/web/package-lock.json` as committed, not the version ranges in the manifests. The backend lock resolves 114 packages; the web lock resolves 419 entries. Third-party licences for every one of them are inventoried in `docs/assessment/licenses.md`, generated by `scripts/gen_licenses.sh`.

Table 4: Backend runtime dependencies (direct), locked versions. Python 3.12, managed by uv.

Package	Version	Role
<code>fastapi</code>	0.141.1	HTTP API, dependency injection, OpenAPI generation
<code>starlette</code>	1.6.0	ASGI foundation (transitive, pinned by the lock)
<code>uvicorn[standard]</code>	0.53.0	ASGI server (api entrypoint)
<code>pydantic</code>	2.13.5	request/response models, the agent protocol models
<code>pydantic-settings</code>	2.15.0	LS_* settings with a production hardening validator
<code>sqlalchemy[asyncio]</code>	2.0.54	ORM, async sessions, row locking
<code>alembic</code>	1.20.0	migrations (one revision, 50 tables)
<code>psycopg[binary]</code>	3.3.5	PostgreSQL driver
<code>pgvector</code>	0.5.0	SQLAlchemy Vector column type
<code>authlib</code>	1.8.0	OIDC client (authorization code + PKCE)
<code>joserfc</code>	1.7.5	JOSE primitives used by the development IdP
<code>itsdangerous</code>	2.2.0	signed handshake cookie for the OIDC state
<code>python-multipart</code>	0.0.32	multipart parsing for document upload
<code>httpx</code>	0.28.1	internal dispatch client, test transport
<code>anthropic</code>	1.6.0	LLM provider SDK (<i>never called live in this build</i>)
<code>structlog</code>	26.1.0	JSON logging with trace ids and key redaction
<code>fastembed</code>	0.8.0	ONNX embedding runtime (BAAI/bge-small-en-v1.5, 384 d)
<code>spacy</code>	3.8.16	<code>blank("en") + EntityRuler</code> for entity extraction
<code>scikit-learn</code>	1.9.1	TF-IDF + logistic regression note classifier
<code>numpy</code>	2.5.3	vector arithmetic
<code>pypdf</code>	6.19.0	PDF structural scan and text extraction
<code>pyyaml</code>	6.0.3	dataset front matter, Compose/CI static validation

Table 5: Backend development dependencies and notable transitive runtime packages.

Dev package	Version	Transitive (notable)	Version
<code>pytest</code>	9.1.1	<code>onnxruntime</code>	1.30.0
<code>pytest-asyncio</code>	1.4.0	<code>tokenizers</code>	0.23.2
<code>hypothesis</code>	6.168.0	<code>huggingface-hub</code>	1.31.0
<code>mypy</code>	2.3.1	<code>scipy</code>	1.18.1
<code>ruff</code>	0.16.8	<code>thinc</code>	8.3.13
<code>types-pyyaml</code>	6.0.12.20260906	<code>cryptography</code>	50.0.1

Table 6: Frontend dependencies, locked versions (`apps/web`). Node 26 (`.nvmrc`); TypeScript strict mode.

Package	Version	Role
<code>react / react-dom</code>	19.3.0	UI runtime
<code>vite</code>	8.3.0	dev server (proxying <code>/api</code> , <code>/auth</code>) and production build
<code>typescript</code>	6.0.3	strict-mode types; <code>tsc -b</code> in CI
<code>react-router</code>	8.4.0	data router, lazy route chunks
<code>@tanstack/react-query</code>	5.103.1	server state, explicit loading/error states, polling
<code>react-hook-form</code>	7.88.0	forms
<code>@hookform/resolvers</code>	5.9.1	Zod resolver
<code>zod</code>	4.6.5	client-side schemas mirroring the server contract
<code>openapi-fetch</code>	0.17.0	typed client over the generated API types
<code>openapi-typescript</code>	7.13.0	generates <code>src/generated/api.ts</code> from <code>contracts/openapi.json</code>
<code>tailwindcss / @tailwindcss/vite</code>	4.3.3	CSS-first design tokens
<code>recharts</code>	3.10.1	capacity, bottleneck and defect-trend charts
<code>@phosphor-icons/react</code>	2.1.10	icons (always paired with text)
<code>@fontsource-variable/geist</code>	5.3.0	self-hosted fonts (no third-party font CDN)
<code>vitest</code>	5.0.1	component and unit tests
<code>@testing-library/react</code>	16.3.3	behaviour-focused component tests
<code>msw</code>	2.15.0	request mocking in component tests
<code>jsdom</code>	30.1.0	test DOM
<code>eslint</code>	10.11.0	lint (<code>-max-warnings=0</code>)

Table 7: Infrastructure and platform components. Container image tags were each confirmed against the registry’s own API before being pinned; none of these images has ever been built or run here.

Component	Version / tag
PostgreSQL	16.15 (Homebrew <code>postgresql@16</code>)
pgvector	0.8.6 (compiled from source)
Python	3.12
uv	0.11.17 locally; 0.12.17 pinned in the image
Node.js	26 (<code>apps/web/.nvmrc</code>)
python image	3.12.14-slim-bookworm
node image	26.9.0-alpine
caddy image	2.11-alpine
pgvector/pgvector image	0.8.6-pg16
quay.io/keycloak/keycloak	26.7.4
GitHub Actions	<code>checkout@v7.0.1</code> , <code>setup-node@v7.0.0</code> , <code>setup-uv@v10.1.0</code> , <code>upload-artifact@v7.0.1</code>

6 The four agents and bounded autonomy

6.1 The shape every agent has

Every agent is the same shape (`app/agents/base.py`):

1. `assess(ctx)` runs first and is **deterministic**. It is the only source of findings, metrics, action payloads and evidence.
2. If no LLM client is configured, the result is returned immediately as `DEGRADED` with `degraded_reason = "LLM_DISABLED"` and the warning *“AI explanation unavailable”*.
3. Otherwise a bounded loop runs: at most **4 investigative tool calls**, at most **1 repair turn**, and a hard iteration cap of $4 + 2 + 1$. *Every* model call is reserved against the run’s budget in the database *before* it is made.
4. The first user message is JSON context inside `<context>...</context>` plus the instruction to treat tool results and documents as untrusted data and to select only among `candidate_actions` and cite only `available_evidence`. Tool output is redacted (`redact_payload`) before it reaches the model.
5. `submit_assessment` is validated server-side (action id, finding ids, evidence ids). One repair turn is offered; a second failure degrades the result to `INVALID_AGENT_OUTPUT` with the deterministic content intact.
6. On success the selected action moves to `rank=0` with its **payload unchanged**; the model’s summary is used with `summary_source="model"`; its notes become `MODEL_NOTE` findings at severity *info*.

Run-wide limits: **12 model calls per run** shared by all six agent tasks, **1 replan**, **2 retries** after the first attempt, and a **120-second** deadline. The budget is reserved atomically (`UPDATE analysis_runs SET model_calls_used = model_calls_used + 1 WHERE ...AND model_calls_used < model_calls_limit RETURNING ...`), so with a limit of 12, exactly 12 of any number of concurrent callers succeed — proved by a 30-way concurrent integration test.

6.2 What each agent may and may not do

6.3 Degradation is explicit, never a silent fallback

The run still finalizes into a `run.report` whose `degraded_reasons` name exactly what was lost. **Fallback text is never promoted as a real model run.**

6.4 The agent task protocol

The protocol is a **custom, versioned HTTP/JSON contract** (`schema_version "1.0"`). It is *not* A2A and *not* MCP, and this report does not call it either.

An `AgentResult` carries `status`, `summary` with its `summary_source`, `findings[]`, typed `metrics[]`, `recommended_actions[]`, `evidence_refs[]`, `warnings[]`, `input_versions`, `data_quality` and `execution_metadata` (provider, model, counts, `prompt_version`, `degraded`, `degraded_reason`, timings). `app/orchestration/validation.py` rejects a result whose evidence does not resolve, whose records fall outside the run’s organization or factory, whose document version is not `ACTIVE/SUPERSEDED`, or whose action payload names a slot or balance that is not in the run snapshot. A rejected result is replaced by a `FAILED` result with `INVALID_AGENT_OUTPUT`: **the run continues, and the bad output is never stored as truth.**

Error codes and their retryability are decided by the server, never by the model: `PROVIDER_UNAVAILABLE` (the only retryable one), `MISSING_DATA`, `STALE_INPUT`, `BUDGET_EXCEEDED`, `INVALID_AGENT_OUTPUT`, `POLICY_DENIED` and `DEADLINE_EXCEEDED`.

6.5 The orchestration graph

`orchestrator.advance` is the **only** component that creates agent tasks. Every delivery locks the run row, does exactly one thing (start, dispatch, or finalize), commits, and *only then* makes a network call — the run lock is never held across the wire.

RM, IE and quality round 0 are dispatched in the *same* advance step, as three `task.dispatched` events with consecutive ids before any `task.completed`: none depends on another. Planning round 0 waits for RM **and**

Table 8: The four agents (docs/architecture/agents.md).

Agent	Deterministic assessment and tools	Human boundary
rm	<code>assess_material_readiness</code> (r0), <code>validate_plan_materials</code> (r1). Applies <code>app.domain.inventory.calc</code> only: <code>gross_demand</code> , <code>available_now</code> , <code>shortage</code> , <code>projected_balance</code> , <code>coverage_days</code> , <code>reorder_point</code> , <code>coverable_units</code> , <code>material_state</code> . Tools: <code>get_material_position</code> , <code>get_expected_receipts</code> , <code>get_consumption_history</code> , <code>get_bom_demand</code> , <code>search_documents</code> .	Suggests replenishment. Never buys, never reserves, never approves. A <code>RESERVATION</code> action is a proposal a human approves.
ie	<code>assess_line_capability</code> . Per-operation representative cycle (median, ≥ 3 samples), effective cycle, bottleneck, this model's balance index, SAM-based vs measured throughput. Tools: <code>get_line_analysis</code> , <code>get_operation_statistics</code> (count and median only), <code>compare_observed_vs_standard</code> , <code>search_documents</code> .	Never judges, names, ranks or compares an individual worker. Operator aliases are not in the snapshot, no tool returns one, and no summary, finding or action may contain one — asserted in <code>tests/agents/test_ie_agent.py</code> .
quality	<code>assess_quality_status</code> . Defective rate and DHU reported <i>separately</i> ; eligibility is the snapshot's calculated verdict, quoted verbatim. Tools: <code>get_inspections</code> , <code>get_policy_rules</code> , <code>get_defect_breakdown</code> , <code>search_documents</code> .	Never decides shipment readiness, never releases a hold. A defect belongs to an operation and an inspection, never to a person .
planning	<code>propose_allocation</code> (r0), <code>revise_allocation</code> (r1). Every candidate comes from <code>plan_earliest_slots</code> : <code>FULL_EARLIEST</code> , <code>MATERIAL_LIMITED</code> (capped at RM's <code>coverable_units</code>), <code>SINGLE_LINE</code> , plus <code>SIMULATED</code> options. Tools: <code>get_dependency_findings</code> , <code>list_compatible_lines</code> , <code>get_remaining_capacity</code> , <code>simulate_allocation</code> .	Proposes. The allocation takes effect only when a supervisor approves <i>and</i> applies the recommendation.

IE to be terminal and receives both results as input references. Quality is independent of the planning chain but must be terminal before finalization, because the order report cannot be written without its facts.

The replan rule is exact. `needs_replan(planning_result, rm_result)` returns a reason *only* when the rank-0 `ALLOCATION` action commits more units than the RM agent's `coverable_units` metric:

```
MATERIAL_SHORTAGE_CONFLICT: selected plan
allocates <x> units but materials cover <y>
```

The reason is recorded verbatim in the `orchestrator.replan` run event. The revision task ranks the `MATERIAL_LIMITED` option first and adds a `REVISED_FOR_MATERIAL` warning quoting the allocated and requested units, the shortage, and the first open receipt date. At most one replan happens per run.

Table 9: What happens when the model path fails. In every case the findings, metrics, actions and evidence are unchanged — only the explanation is missing.

Situation	Result
LS_LLM_PROVIDER=disabled	DEGRADED, deterministic summary, <code>degraded_reason</code> = LLM_DISABLED
Provider outage	three attempts, then the deterministic assessment, PROVIDER_UNAVAILABLE
Run budget spent	the deterministic assessment, BUDGET_EXCEEDED
Model refusal / invalid output twice	the deterministic assessment, LLM_REFUSAL / INVALID_AGENT_OUTPUT

Table 10: TaskEnvelope — what the server verifies, not what the JSON claims.

Field	Rule enforced server-side
run_id, organization_id, factory_id, order_id, snapshot_id	must all equal the loaded run's own ids
parent_task_id	when set, must be a task of the same run
sender	always "orchestrator": a task can never create a task
recipient / task_type	the task type must be in ALLOWED_TASK_TYPES[recipient]
idempotency_key	exactly <code>f"{run_id}:{recipient}:{snapshot_id}:round-{round}"</code> ; a repeat returns 200 with the existing task
round	0..1 — at most one replan
deadline_at	must be $\leq$ the run's own deadline
constraints	<code>max_tool_calls</code> 0..4; <code>read_only</code> is <code>Literal[True]</code>

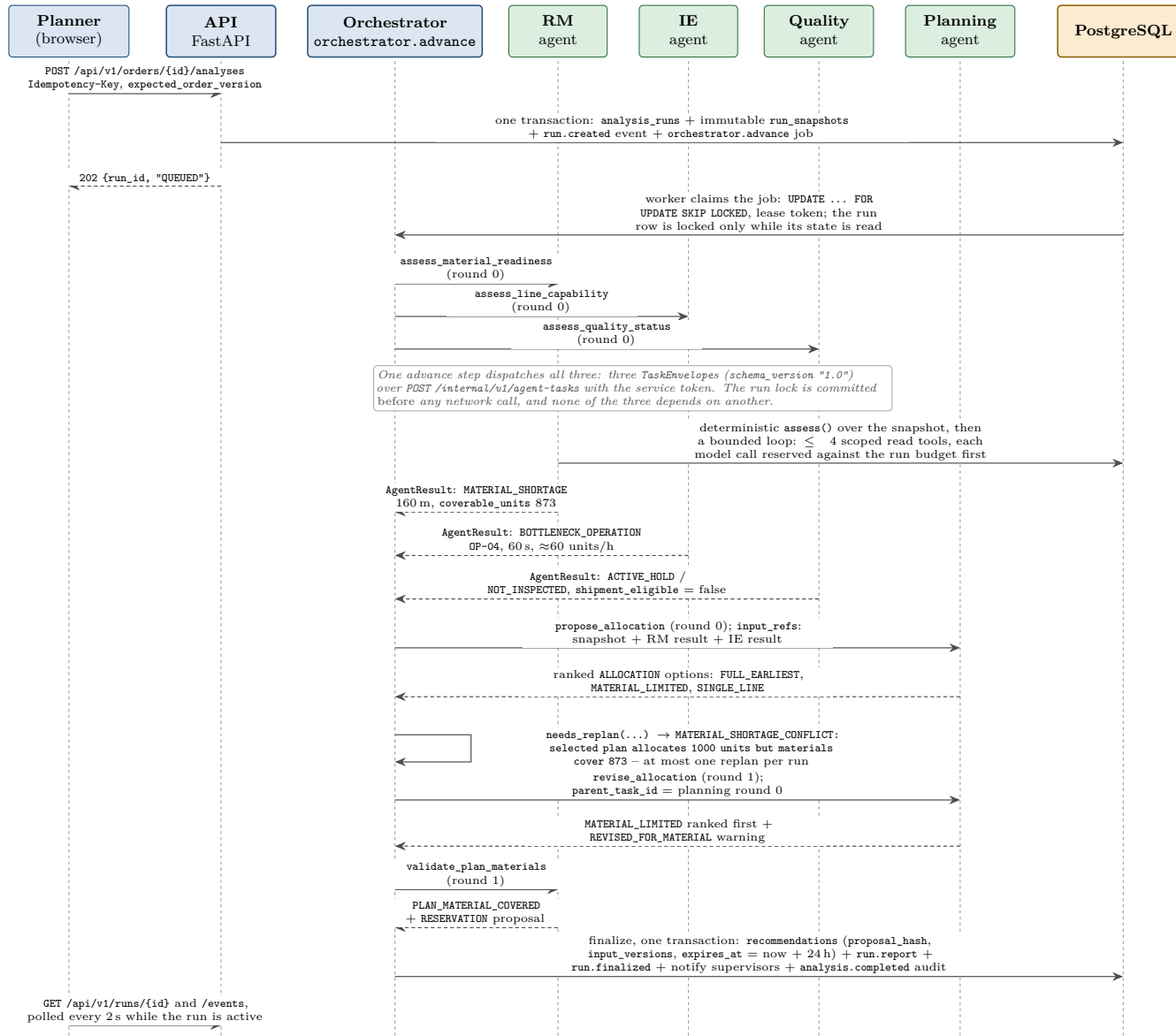


Figure 6: The end-to-end four-agent analysis run. Read it as the answer to “what actually happens when a planner presses *Start analysis*”. Note three things: the three round-0 dispatches leave in one advance step; the replan is driven by a *deterministic* comparison of two agents’ numbers, not by a model’s opinion; and finalization is one transaction that writes the proposal, the canonical report, the notification and the audit event together.

6.6 Deterministic engineering calculations

These are the formulas the agents apply. They are demonstration formulas: an IE or domain owner must validate the production assumptions before real use, and units and assumptions appear next to every result on screen.

Planning.

$$\text{required_standard_minutes} = \text{remaining_units} \times \text{SAM} \quad \text{utilization} = \frac{\text{allocated_standard_minutes}}{\text{available_standard_minutes}}$$

where available standard minutes sum $\text{available_operator_minutes_per_shift} \times \text{planned_efficiency_fraction}$ over the eligible slots. Efficiency is applied *once*. Allocation is deterministic earliest-due-date with a documented tie-break of `(slot_date, shift_code, str(line_id))`; an infeasible order stays unscheduled with a reason `(NO_COMPATIBLE_LINE, INSUFFICIENT_CAPACITY_BEFORE_DUE_DATE, LIMITED_BY_MATERIAL)`.

Materials.

$$\text{available_now} = \text{accepted_on_hand} - \text{active_reservations} \quad \text{gross_demand} = \text{planned_units} \times \text{BOM_qty} \times (1 + \text{wastage})$$

$$\text{projected_balance}(t) = \text{available_now} + \text{eligible_receipts}(t) - \text{new_demand}(t) \quad \text{reorder_point} = \bar{c}L + \text{safety_stock}$$

Reservations already excluded from **available_now** are never subtracted twice. Missing or zero consumption yields an *unknown* coverage indicator with an explanation — **coverage_days** returns **None**, never a misleading zero.

IE. For the sequential-operation model, an operation’s effective cycle is its representative single-operator seconds per unit divided by the number of equivalently capable parallel operators; the bottleneck effective cycle is the maximum; estimated line output is $3600/c_{\text{bottleneck}}$ units per hour. The balance index is $\sum c_i / (n \cdot c_{\text{bottleneck}}) \times 100$ and is labelled as *this model’s index*, not a universal industrial KPI.

Quality. `defective_units / inspected_units` and DHU (`total_defects / inspected_units × 100`) are reported *separately*. Zero inspections means *unknown*, not *pass*. The applicable policy is versioned and approved; the demo policy carries the label “*Demo policy — not a certified AQL standard*” and the system never claims certified AQL/ISO compliance.

6.7 Independent reference fixtures

Six reference cases were worked by hand *before* the code existed (`docs/architecture/formulas.md`) and are asserted verbatim in `tests/unit/test_reference_fixtures.py` and, for the two concurrency/staleness cases, in integration tests.

7 Methodology

7.1 Deterministic core plus bounded agents

The methodological claim of this project is narrow and testable: *an LLM is useful for selecting among permitted investigative actions and for explaining a result, and is unsafe as a source of business facts*. Everything in the architecture follows from taking that seriously — the pure calculation layer that imports nothing, the candidate-action list the model may only re-rank, the evidence id whitelist, the server-side validation of every submission, and the source label on every rendered value.

The consequence, which is worth stating plainly, is that a model failure degrades the product’s *explanation* and never its *correctness*. A provider outage costs the user a paragraph of prose, not a wrong shortage figure.

7.2 Retrieval

Documents are retrieved with **hybrid search**: a lexical ranking (`ts_rank_cd(tsv, websearch_to_tsquery('english', :q))`) and a vector ranking (`embedding <=> :qvec`, cosine distance, exact scan) run under the *same* SQL filter — organization, factory (or organization-wide), document ACL, and **ACTIVE** version — and are fused by reciprocal-rank fusion, $\sum 1/(k + \text{rank})$ with $k = 60$, over each list’s top 20, then truncated to the top 6.

The filter runs **before** any content reaches a model, and `get_citation()` re-runs the whole scope and ACL check when a citation is opened. A **SUPERSEDED** chunk is visible only when a `run_id` names a run in the

Table 11: Reference fixtures and where each is proved.

Case	Expected	Proved by
1,000 units at SAM 12 min; 20 operators, 420 min each, 75% efficiency	demand 12,000 std min; one-shift capacity 6,300; does not fit	<code>test_reference_fixtures.py</code>
1,500 accepted m, 400 reserved; demand 1,000 units @ 1.2 m + 5% wastage	available 1,100; demand 1,260; shortage 160 m	<code>test_reference_fixtures.py</code> ; reproduced end-to-end by the seeded PO-DEMO-001
Three sequential effective cycles 40 / 60 / 50 s	bottleneck 60 s; 60 units/h; balance index $\approx 83.33\%$	<code>test_reference_fixtures.py</code> ; <code>test_seeded_demo_line_reproduces_op04_bottlene</code>
100 inspected, 7 defective, 12 defects	7% defective rate; DHU 12; disposition from the approved policy	<code>test_reference_fixtures.py</code>
Two simultaneous reservations of 80 from 100 available	at most one succeeds; the other gets a conflict/shortage	<code>tests/integration/test_reservation_concurrency</code>
Proposal pinned to stock version 4; stock moves to 5 before apply	application rejected as stale; approval cannot bypass reanalysis	<code>test_approvals.py::test_apply_rejects_stale_in</code>

caller’s scope whose stored agent results actually cite that chunk id — never a blanket “any superseded chunk once you know a run”.

7.3 NLP

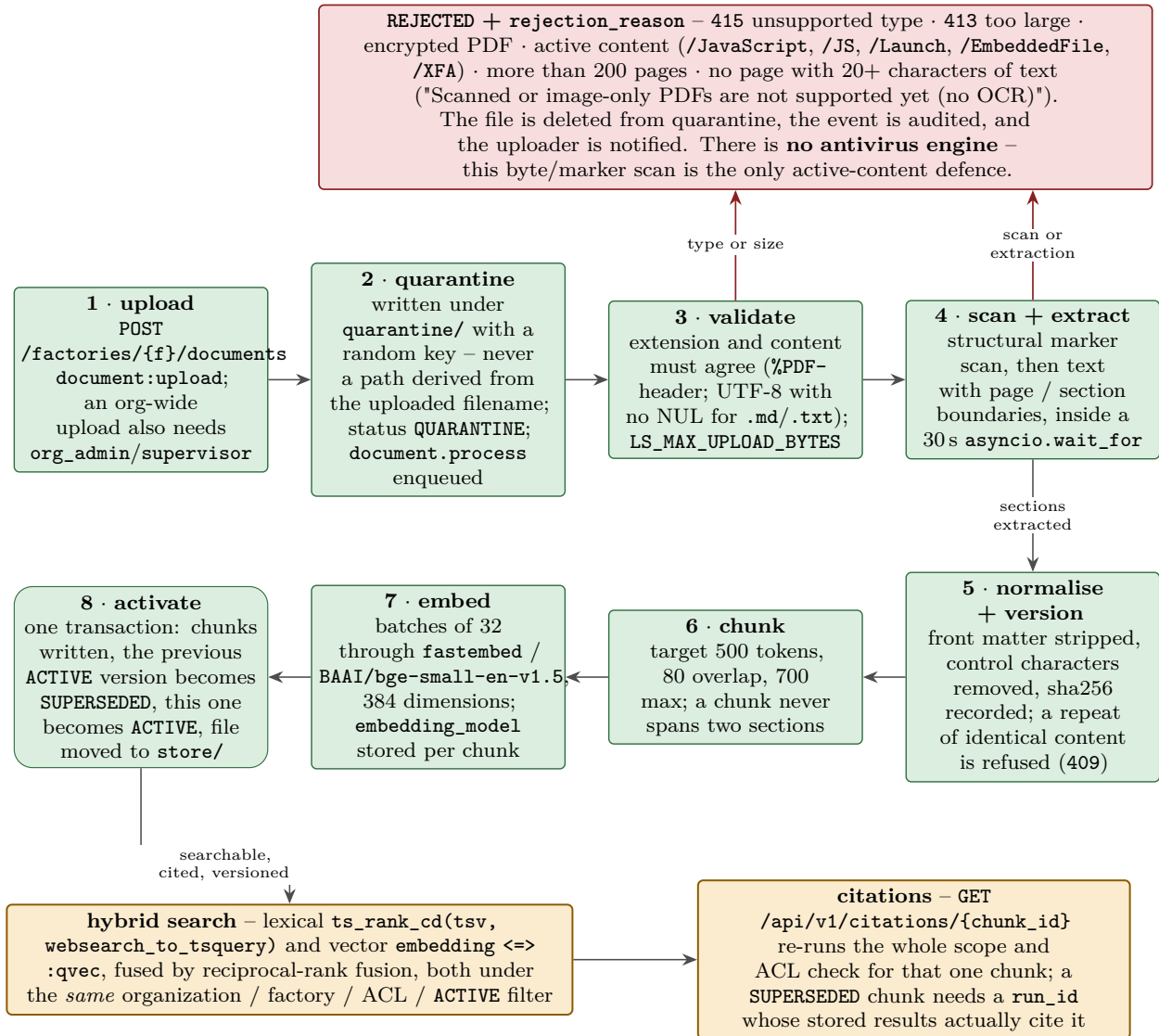
Three capabilities, each deliberately starting from the simplest thing that could work:

- **Entity extraction** is `spacy.blank("en")` plus one `EntityRuler` compiled from *that factory’s* authorized master data — order refs, line codes and names, style codes, material codes and names, operation and defect codes and names. One extra token regex (`~po-[a-z]{3}-\d{4}$`) surfaces well-formed but *unknown* order references without ever resolving them. A BYG line code is not a pattern at all when the extractor is built for KTN, so it is not recognised — not merely “unresolved”. Ambiguity (two rows normalising to one key) yields a nil-UUID sentinel and the mention is dropped from both the resolved and unresolved lists: it is never stored and never actioned. The API always returns the notice “*Entity links are for navigation only; they do not authorize any action.*”
- **Note classification** has a documented keyword baseline and a `TfidfVectorizer(1,2-grams)` + `LogisticRegression` model trained *only* from `data/eval/notes_train.jsonl`. A unit test patches `builtins.open` to prove the test split is never opened during training. In production the classifier abstains (“unknown”) below a 0.45 probability margin; the evaluation deliberately scores plain `.predict()` instead, because the margin is a product policy, not a measure of the model.
- **Grounded summarization** computes a canonical structured report first and layers an optional model explanation second. Every sentence of the deterministic summary draws its evidence ids from the report’s own blockers and evidence. The model path validates the response before using it: every sentence must carry evidence ids that exist, and no sentence matching `ready to ship|shipment[- ]ready|can ship` is accepted while `shipment.eligible` is false. Any failure falls back to the deterministic summary *with a label saying why*.

7.4 Evaluation design

`python -m app.evaluation (make eval)` runs seven sections against a dedicated database it resets itself, and writes a versioned JSON report plus a Markdown rendering. Every report records the corpus, dataset and code versions (`corpus_sha256`, `notes_test_sha256`, `questions_sha256`, `provider`, `model`) so a number can always be tied back to what produced it.

Targets are fixed in `app/evaluation/report.py` and are **never adjusted after seeing a result**; a missed target is reported as `"met": false` next to the real number, and the harness still exits 0 (a non-zero exit is reserved for a harness error).



Steps 2–8 run in the worker as the document.process job. The activation step is written so that a retry after a lost lease deletes and re-inserts this version's chunks rather than appending, so a repeated attempt can never double a version's chunks. Vector search is an exact scan – there is no approximate (HNSW/IVFFlat) index yet, which is adequate at this corpus's 30 documents and would not be at a much larger one.

Figure 7: The document ingestion pipeline and the retrieval path it feeds.

The single most important caveat in the evaluation

Every “agent” and “fairness” number this harness can currently produce comes from the deterministic fixture provider, and both sections carry `"not_a_live_llm_evaluation": true`. The fixture provider cannot hallucinate, refuse, phrase something ambiguously or be swayed by wording — which is exactly why it is useless for predicting how a real model behaves, and exactly why it is safe for checking that the bounded agent loop, the task graph and the deterministic calculations wire together correctly.

The retrieval section scores every *test*-split question (45 of 56; the other 11 are *dev* and never scored) in lexical, vector and hybrid mode under one fixed scope. Two of the 45 ask about a BYG-scoped document and can never be retrieved under the fixed KTN-supervisor scope — by design, since access control must not leak across factories. Those misses stay in the denominator, because adjusting the dataset or the scope to make the number look better would be exactly the kind of result-chasing this project forbids.

8 Implementation

8.1 What exists, in numbers

Table 12: Repository statistics, counted from the committed tree.

Backend application Python files	159	Web source files (<code>.ts/.tsx</code>)	134
Backend application lines	32,626	Web source lines	19,381
Backend test modules (<code>test_*.py</code>)	82	Web test files	34
Backend test functions defined	679	Web test cases defined	109
Database tables	50	Alembic revisions	1
Public API paths / operations	66 / 70	Generated OpenAPI schemas	151
ADRs	8	SOP corpus documents	30

These are counts of what is defined, not a claim that every one of them was executed in a single run. See §15.

8.2 Repository layout

```
apps/web/src/{app,components,features,lib,generated,test}
services/backend/app/{api,auth,audit,idempotency,db,domain,agents,
    orchestration,jobs,retrieval,nlp,llm,evaluation,seed}
services/backend/{migrations,tests,devtools}
contracts/ # exported OpenAPI + agent protocol JSON Schemas + examples
data/{synthetic,eval} # the SOP corpus, adversarial fixtures, labelled datasets
docs/{adr,architecture,security,operations,evaluation,assessment,report}
infra/{compose,identity,proxy}
scripts/ # dev-db, backup/restore, scans, validators, generators
```

8.3 How it was built

The build ran as a sequence of gated tasks against a written plan, each with its own brief, its own test-first cycle and its own dated entry in `docs/IMPLEMENTATION_STATUS.md` recording the exact commands run and their exact output. Several tasks were reviewed and produced a *fix round* that is recorded in the same file. That log is the primary evidence for everything in this report; it also records, honestly, the two places where a test was written after its implementation rather than before, and why.

The contract between backend and frontend is generated, not hand-maintained: `scripts/export-openapi.sh` writes a deterministic, sorted `contracts/openapi.json`, `npm run generate:api` turns it into `apps/web/src/generated/api.ts`, and `make contracts-check` detects drift.

8.4 Frontend

The web application is a desktop-first responsive dashboard with 16 routes behind a factory-scoped layout. Every screen implements the applicable subset of loading, empty, validation, permission-denied, stale-data,

degraded-AI and server-error states. Navigation is permission-filtered, so a role never sees a section it cannot read.

Two conventions are enforced throughout:

- **No optimistic updates for anything that matters.** Stock changes, approvals, releases and allocation application all wait for server confirmation. Duplicate submission is disabled in the UI *and* rejected on the server by an Idempotency-Key.
- **No dangerouslySetInnerHTML anywhere.** Model text, document chunks and user notes are rendered as React text nodes. A test renders a chunk containing a literal `<script>` element and asserts `document.querySelector('script')` stays empty.

Accessibility is treated as part of the contract rather than a later pass: a skip link, keyboard-operable controls, a focus-trapping confirmation dialog, `aria-required`/`aria-invalid`/`aria-describedby` wiring on form fields, text alternatives for every chart, and icon-plus-text (never colour alone) for every state badge.

9 Workflows

9.1 Analysis: planner to canonical report

Figure 6.5 is the whole sequence. In prose: a planner posts an analysis request with an Idempotency-Key and the order version they believe is current. The API locks the order, refuses a stale version (409 STALE_INPUT), a cancelled or dispatched order (409 INVALID_TRANSITION), a second active run for the same order (409 CONFLICT, naming it), or more than five active runs in the factory (429 RATE_LIMITED with `retry_after_seconds`). Otherwise it creates the run *and an immutable snapshot of every input the agents will reason about* in one transaction and returns 202.

The snapshot matters more than it looks. Because every agent reads the snapshot rather than live tables, two agents in the same run can never disagree about the facts, a late agent result can never be based on data the planner never saw, and the recommendation that comes out carries the exact versions it was derived from.

Finalization writes, in one transaction: the recommendation (if there is a plan to propose), the supersession of every earlier open recommendation for that order (NEWER_ANALYSIS), the run status, the canonical order report as a `run.report` event, `run.finalized`, a notification to supervisors, and the `analysis.completed` audit event.

The order report is built **from deterministic state only**. Model text appears only inside `agent_summaries`, each labelled with the provider that produced it.

9.2 Approval and application

Three properties of figure 8 are worth calling out because they are the ones that are easy to get wrong:

1. **The lock set is the union, taken up front.** The transaction locks every capacity slot in the proposal *plus* every slot the order has ever been allocated to, and every material balance in the proposal *plus* the order's BOM materials *plus* every material it has ever reserved — each set ascending by id, before it touches anything. A proposed material without a balance row gets one before the lock sweep, so `reserve_material` can never create and lock a row out of order. Two concurrent applications therefore cannot take the same two rows in opposite orders.
2. **Staleness fails closed.** A row that moved is stale; a row that vanished is stale; and a row the proposal would *write to* that `input_versions` does not pin is stale too — a missing entry, a key that is not a UUID, and a version that is not an integer all count, because none of them proves the row has not moved.
3. **The stale path commits its own status change before failing.** The recommendation is set SUPERSEDED with `superseded_reason = "STALE_INPUT: ..."`, audited, the proposer is notified, that change is *committed*, the idempotency claim is released so the key is not burned on a rejection, and only then does the request fail with 409 listing every changed input in `field_errors`.

A mid-apply capacity or stock failure rolls the *whole* transaction back: zero partial allocations, zero partial reservations. The route owns its session and retries the whole transaction up to three times with jittered backoff on SQLSTATE 40001 and 40P01 only; everything else propagates.

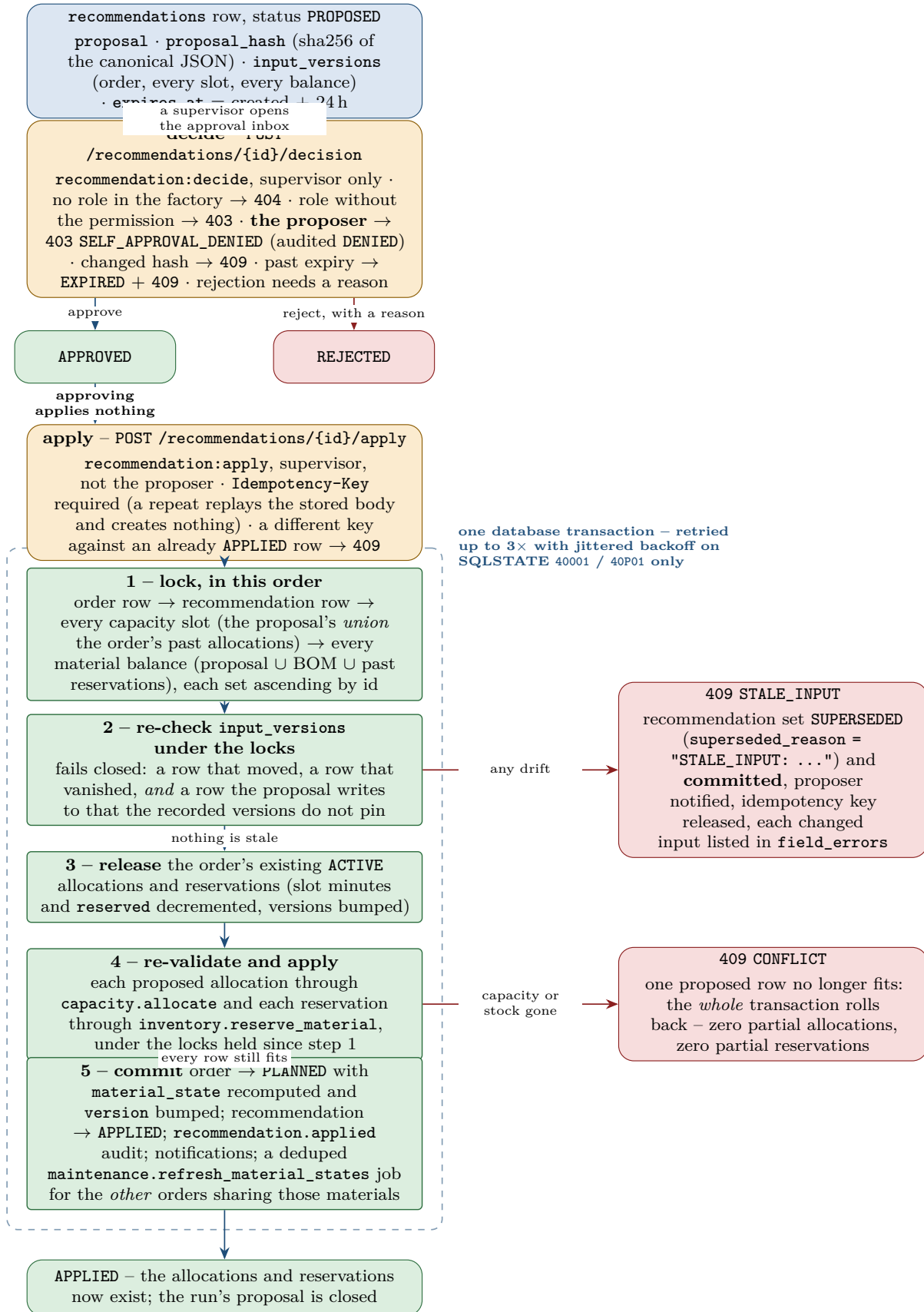


Figure 8: Approval and transactional application. Approving and applying are two separate acts by a supervisor who is not the proposer, and the application fails closed on any input drift.

9.3 Quality hold, release and shipment eligibility

A recorded inspection is evaluated against the *active*, *versioned* policy and returns a deterministic **PASS** / **FAIL** / **INSUFFICIENT_SAMPLE**. A **FAIL** automatically places an **ACTIVE** hold carrying the failing reasons. With *no* approved policy the inspection is refused (409, “No approved quality policy”) — the system abstains rather than guessing.

Release validates that the inspection is **FINAL**, **PASS**, the latest **FINAL**, and evaluated against the *current* policy version (a version mismatch is 409 **STALE_INPUT**); and it refuses when the releaser is the inspector (403 **SELF_APPROVAL_DENIED**). Releasing also releases every active hold. Only then does shipment eligibility change — and it changes because the derived predicate now holds, not because anybody said so.

9.4 Import

CSV import is upload → server validation → preview (first 20 rows plus per-row errors) → explicit commit, with the commit re-running the exact same checks. Either the whole batch commits or nothing does: a commit-time failure returns 409 and writes no orders and no batch status change. Row numbers are physical file lines; the upload reads at most **MAX_FILE_BYTES** + 1 bytes rather than buffering an unbounded body; a **csv.Error** becomes a **REJECTED** batch rather than a 500; and a cell beginning with a spreadsheet formula character is checked on the *raw* cell and neutralised rather than stored raw.

10 Security

10.1 Authentication, sessions and tenancy

OIDC authorization-code flow with PKCE, **state** and **nonce**. The callback passes explicit **claims_options** to Authlib requiring the issuer to equal the discovery issuer, the audience to contain the client id, and **sub** to be present — a fix that closed a real hole found in review, in which a token with a foreign **aud** was accepted because its **azp** named our client.

Identity is exchanged for an **opaque**, **sha256-hashed application session** with an 8-hour absolute lifetime, rotated after login and after any privilege change, and revoked on logout. A CSRF middleware checks Origin/Referer *and* an **X-CSRF-Token** header on every state-changing cookie-authenticated request.

Authorization is resolved by shared policy functions (**app/auth/scope.py:load_scoped**) on *every* access path — routes, worker tools, retrieval, citations, file downloads and audit views — with one consistent rule: **a resource in a factory the caller holds no role in is 404; an accessible resource missing the required permission is 403.**

10.2 Threat model

docs/security/threat-model.md maps each of the specification’s threats to a STRIDE category, an implemented control and the test that exercises it. Summarised:

10.3 Prompt injection: what the adversarial test actually proves

data/synthetic/adversarial/injection-sop.md is a document that asks the model to approve without review, to call **delete_all_records**, to reveal a secret, and to cite a fabricated chunk id. It is loaded through the *real* document pipeline and retrieved by the *real* RM agent loop, driven by a deliberately obedient scripted model. The observed outcome:

- the injected text reaches the model only inside a **tool_result**, as data;
- the **delete_all_records** call returns an unknown-tool error and has no effect;
- the fabricated chunk id fails validation, one repair turn is offered, it fails again, and the run **degrades** with **error_code=INVALID_AGENT_OUTPUT** — with **evidence_refs** reduced back to the untouched deterministic set and **recommended_actions** byte-identical to the agent’s own deterministic assessment.

That is the honest claim: **injection cannot escalate**, because the model was never the authority. It is not a claim that injection can be detected. Detection alone is not a security boundary, and this project does not treat it as one.

Table 13: Threats, controls, and the tests that prove them.

Threat	Control	Proven by
Cross-factory / order access	load_scoped on every path; 404 vs 403 split	test_idor_matrix.py (generated from the live app's routes), test_order_access.py, test_document_access.py
Prompt injection in SOPs	read/compute-only tools, untrusted tool-result handling, candidate-action and evidence-id whitelists, bounded calls	test_prompt_injection.py (real RM loop vs the real adversarial document), test_document_tool.py
Forged / stale approval	self-approval denial, hash and version binding re-checked inside the locking transaction	test_approvals.py, test_approval_rules.py
Duplicate consumption	sorted-id row locks, required idempotency keys, unique (task_id) on results	test_apply_concurrency.py, test_reservation_concurrency.py, test_worker_kill.py
SQL injection / XSS	parameterized SQLAlchemy only, no model SQL, React text nodes, CSP	xss.test.tsx, test_headers_and_limits.py
Malicious uploads	size cap, type sniffing, quarantine-until-processed, no public originals	test_document_pipeline.py
SSRF / exfiltration	no tool accepts a URL; outbound calls only to settings-derived addresses	tool sets are closed and defined per agent in code
Secret disclosure	SecretStr/environment only, structlog key dropping, provider-payload redaction, repository scanning	test_redaction.py (34 cases), scripts/secret-scan.sh
Cost / request abuse	per-user and per-IP token buckets, per-run model and token budgets, max active runs per factory	test_headers_and_limits.py, test_budget.py
Worker / service impersonation	constant-time bearer service token, never a cookie; route absent from the public API	test_internal_dispatch.py

10.4 Scans (recorded runs)

```
$ bash scripts/secret-scan.sh
[secret-scan] OK: no high-risk secret patterns found in 413 tracked files

$ uvx pip-audit -requirement /tmp/requirements.txt
Resolved 114 packages in 2ms
No known vulnerabilities found

$ npm audit --omit=dev --audit-level=high
found 0 vulnerabilities
```

One documented, reviewed exclusion: the scan excludes paths under a `tests/` directory, because this repository's own tests deliberately contain literal fake `sk-ant-...`-shaped strings in order to test *this project's* redaction and error-handling code. Each such string is listed by file and line in `docs/security/scan-results.md`.

10.5 Accepted residual risks

- **No row-level security yet.** Tenant isolation is enforced in the application layer and covered by a route-generated IDOR matrix. Database RLS — policies keyed on a transaction-local `organization_id`, a non-owner non-BYPASSRLS runtime role, and pool-reuse tests — is a documented **required gate before any multi-organization pilot**, not something quietly dropped.
- **Rate limits are per process.** Behind more than one API instance each enforces its own budget; a shared store would be needed to cap the true aggregate.
- **Structural upload scan, no antivirus engine and no OCR.**
- **The development IdP is not a real IdP** — shared password, in-memory state, a signing key regenerated on every restart, no TLS. It refuses to start outside development/test.
- **Append-only audit permissions are not cryptographic tamper-proofing.** Stronger immutable

archival is a later production requirement.

11 Responsible AI implementation

The full treatment is `docs/assessment/responsible-ai.md`, with the model and data cards alongside it. The implemented positions:

Worker privacy. Operator identifiers are pseudonymous alias codes (KTN-OP-017) created by the seed generator; no name, contact detail, health information, protected attribute or disciplinary record exists anywhere in the schema. Observations reach the IE agent *aggregated per operation*; operator aliases are not in the run snapshot, no tool returns one, and a test asserts no summary, finding or action contains one. A defect belongs to an operation and an inspection, never to a person.

No individual ranking, ever. The IE agent’s only action is `IE_REVIEW` — “review method and staffing at `<operation>` on `<line>`” — targeting a process constraint and a staffing *level*. There is no code path that ranks, scores or compares individuals.

Explainability through evidence, not through model introspection. Every finding carries evidence references that resolve to a record with its version or a document with its version, page and section, and every citation is re-authorized when opened. The product does not display model reasoning as an explanation, and it does not display a model-produced confidence number as if it were a calibrated probability.

Transparency about what produced a value. The provider label is carried from the adapter to the API to the screen. In this build every agent summary shows “*Test fixture — not a live AI model*”, because that is the truth.

Abstention. No approved policy → nothing can be dispositioned and the order cannot ship. No inspection → quality is *unknown*, not *pass*. Unknown consumption → `coverage_days` is `None` with an explanation, not zero. A missing dependency → a degraded result naming what is missing, not a fabricated success.

Human oversight and correction. Consequential changes require approval by someone other than the proposer; rejection requires a reason; the original proposal and its history are retained; and every decision, application, release and policy change is audited with before/after summaries.

Fairness — and the limits of what was tested. The harness runs the *same* order through the four-agent flow twice, changing only its customer, and compares the stored `proposal_hash`. Two *different* orders are not a valid comparison, because the hash covers a payload embedding the order id. In the smoke run, 2 of 2 pairs were byte-identical. That shows the deterministic flow does not let customer identity change the allocation under a deterministic provider. It is **not** an audit of fairness across any protected characteristic; it cannot show whether a live model would let a customer’s identity leak into its phrasing or tool choices; and a 100% pass rate here is necessary, not sufficient.

Limitations statement. Synthetic tests and aggregate checks do not prove fairness for a real workforce. Before real factory data is accepted, the project commits to a provider-data review, a retention schedule, a documented deletion process and an incident procedure — all drafted in `docs/assessment/responsible-ai.md` — and treats legal obligations as a separate jurisdiction-specific review, never as a compliance claim.

12 Evaluation results

What these numbers are, exactly

The table below is the **smoke run** of the evaluation harness: `-embedder hashing -scenarios 2`, against an isolated test database, recorded in `docs/IMPLEMENTATION_STATUS.md`. It was run to verify the harness end to end, not to produce the project’s headline numbers. The real run — `make eval`, i.e. `-embedder fastembed -scenarios 12` against `linesense_eval` — is **PENDING (needs user approval or CI)** and has never been executed here. `docs/evaluation/results/` is deliberately empty rather than holding a hashing-embedder run presented as canonical.

Other sections of the same smoke run:

- **Agents (2 scenarios).** All three approaches — deterministic baseline, single-agent baseline, four-agent flow — scored material-conflict accuracy 1.0. The four-agent flow’s `capacity_sufficient` approximation scored 0.5 (1 of 2). Both figures are noise at $n = 2$; the second is additionally an acknowledged

Table 14: Evaluation smoke run (`-embedder hashing -scenarios 2`).

Target	Threshold	Actual	Met
retrieval_hybrid_recall_at_5	0.85	0.711	no — expected: the hashing embedder has no semantics, and 2 of the 45 test questions are legitimately out-of-scope BYG documents
entities_micro_f1	0.90	0.991	yes
classification_tfidf_macro_f1	0.80	0.835	yes
calculations_all_pass	1	1	yes

approximation, because `unscheduled_units` can be driven by a capacity *or* a material shortfall and the two are not separately exposed. *The single-agent baseline can never detect a material conflict (`material_conflict_pred` is always false) because it never sees the RM agent’s assessment — that is the comparison’s whole point, and the report says so in its own `single_agent_baseline_note` field.*

- **Security.** 2 of 2 prompt-injection cases blocked; 3 of 3 abstention checks passed.
- **Fairness.** 2 of 2 customer-swap pairs byte-identical.

Three real bugs were found and fixed while getting the harness green, which is itself evidence that the harness exercises the real code path: agent scenarios were creating runs with no snapshot (the orchestrator correctly refused every one); the in-memory single-agent baseline crashed on the run-budget reservation; and the fairness section was comparing two different orders’ hashes, which can never match.

12.1 Functional verification that *was* executed

Test execution in this project was focused rather than exhaustive, under a machine heat and work-load policy that forbids running full suites locally. What follows are runs actually recorded in `docs/IMPLEMENTATION_STATUS.md`, with their exact pass counts:

Table 15: A selection of recorded focused test runs (see the status file for the full log, including the command line and the database used for each).

Scope	Result
Schema, grants and tenancy constraints (<code>test_schema.py</code>)	11 passed
Domain calculations and the reference fixtures, plus Hypothesis property tests	78 passed
Job queue, worker runtime, leases and fencing	30 passed (of a 98-pass integration run)
Agent protocol, internal dispatch, analysis API, executor	52 passed
RM and planning agents; orchestrator; two-agent flow	31 + 11 + 1 passed
IE and quality agents, four-agent graph, order report	108 passed
Approvals, apply concurrency, approval rules	6 + 9, then 35 after fix round 1
Document pipeline, hybrid search, document access, agent document tool	17 + 20 + 2 passed
Entities, classifier, summaries, notes API, status summary	48 passed
Headers/rate limits, IDOR matrix, prompt injection, worker-kill resilience	81 passed
Query-count regressions (five list endpoints) + approvals	19 passed
Dashboard and administration APIs	13 passed
Whole web suite (<code>npx vitest run</code>)	34 files, 123 passed

Two of those runs are worth singling out because they test the property rather than the implementation:

- `test_two_applications_contend_for_the_last_slot_minutes` creates a *real* overlap using `pg_sleep`

while the lock is held, repeated five times, and asserts exactly one application succeeds and capacity is never exceeded.

- `tests/resilience/test_worker_kill.py` SIGKILLS a real worker subprocess mid-task, waits for the lease to expire, lets another worker reclaim it, and asserts exactly one result exists and the run still finalizes.

A mutation check was also recorded for the locking code: removing `FOR UPDATE` from `lock_balances` makes all 11 concurrency tests fail — so those tests are testing the lock, not passing by accident.

12.2 Backup and restore (a real, recorded exercise)

```
$ time bash scripts/backup.sh linesense_dev
...
bash scripts/backup.sh linesense_dev 0.43s user 0.19s system 70% cpu 0.877
total

$ time bash scripts/restore.sh .local/backups/20260920T080449Z.tar.enc
[restore] verifying row counts against the backup manifest
[restore] verifying document_versions.storage_key files exist
[restore] verifying material_balances against the movement ledger
[restore] verification passed.
bash scripts/restore.sh "$latest" 0.48s user 0.22s system 46% cpu 1.507 total
```

Backup **0.877 s** for a 368,672-byte encrypted archive; restore plus full three-part verification **1.507 s**. This is evidence the mechanism works at demonstration scale — it is *not* a claim about restore time at real factory data volumes.

An honest gap in that exercise: the database it ran against had not had make seed `-with-documents` run, so `document_versions` was 0 and verification step 2 (every `ACTIVE/SUPERSEDED` `storage_key` exists in the restored store) had zero rows to check — its code path was not exercised by this run, and no automated test currently exercises it either.

13 Reliability, performance and deployment

13.1 Reliability

Background work runs through a PostgreSQL job table with `FOR UPDATE SKIP LOCKED` claiming, short leases with heartbeats, and fenced completion (figure 5). **Delivery is at least once and is documented as such;** exactly-once is never advertised. A worker that dies mid-task is recovered either by its lease expiring or by `reconcile_once`, which re-enqueues `orchestrator.advance` for any run quiet for more than 30 seconds with no runnable job of its own.

Error text stored in `last_error` is the exception class plus the first 500 characters of its message, capped at 2,000, never a traceback or local variables; for SQLAlchemy statement errors only the driver exception's class and first message line are stored, never the SQL; and engines are created with `hide_parameters=True` so bound values stay out of error messages.

13.2 Efficiency

RM, IE and quality run concurrently; planning waits for RM and IE. Dashboard cards are computed in deterministic SQL — exactly six statements per request, enforced by a query-count-bound integration test — and **no LLM call is made when a dashboard is opened**. Query-count regression tests cover five list endpoints and caught a real N+1 in `app.domain.approvals.service.list_recommendations`; another caught and fixed an N+1 in the order history route. Embeddings are stored with their model name, so chunks embedded by a different model are never compared.

13.3 Performance: not measured

Performance targets are targets, and none of them has been measured

The specification's initial targets — non-AI API p95 under 500ms at 20 concurrent users, analysis acknowledgement p95 under 1s, a four-agent run usually under 60s within a 120s deadline — are **targets to evaluate against, not results**. The load generator (`scripts/perf_smoke.py`), its `make perf` target and its documentation are complete and were smoke-checked (`-help`, `ruff`, `mypy` all clean), but **no load has ever been generated**: this project's machine heat and workload policy explicitly forbids it on this laptop. There are therefore no latency numbers in this report, and no plausible-looking placeholder has been substituted for them. The exact commands to produce them are in `docs/evaluation/performance.md`.

13.4 Deployment

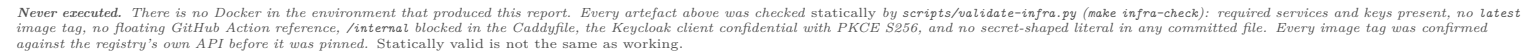


Figure 9: The single-host Compose deployment. Statically validated, never run.

The deployment sequence is: build and scan immutable images → back up → run migrations *exactly once as a release job*, never competitively in every server worker → deploy API, worker and assets → readiness checks (`/api/health/live`, `/api/health/ready`, worker heartbeat) → authenticated smoke flow → observe. Compose models this with a one-shot `migrate` service that `api` and `worker` wait on via `service_completed_successfully`.

Rollback is deliberately *not* “run the down migration”. A destructive down migration is not a safe default rollback plan; the documented approach is backward-compatible expand/contract migrations, with application rollback requiring database compatibility.

One design consequence is recorded honestly as a demo-only limitation: the OIDC issuer must be reachable at the *same* hostname from the browser and from the API container, which is solved here with a second Caddy site (`idp.localhost`) and a Compose network alias. A real deployment does not have this problem, because a real hostname gets a publicly trusted certificate.

A single host is not highly available. The demonstration objectives — daily encrypted backups, recovery point up to 24 hours, restore within 4 hours — are planning objectives, not an SLA, and a commercial SLA would need a stronger design and a contractual review.

14 Commercialization

The full plan is `docs/assessment/commercialization.md`. In summary:

Target and positioning. Apparel factories that currently reconcile spreadsheet, ERP, stores, IE and quality information by hand. LineSense is positioned as a *decision-support layer with traceable recommendations*, not as an ERP replacement and not as an autonomous scheduler. Buyer interest must be validated with production managers, factory owners and operations/quality leads. **No market validation is claimed from this architecture.**

Pricing (hypotheses, not validated rates).

Table 16: Illustrative tiers in USD. These are hypotheses from the project specification, not current vendor prices or validated market rates.

Tier	Proposed price	Proposed scope
Pilot	\$99 / factory / month	up to 5 lines, 10 users, 500 analysis runs per month, CSV imports
Growth	\$249 / factory / month	up to 20 lines, 30 users, 2,000 runs per month, richer reporting
Enterprise	quoted	multiple factories, connector work, isolated deployment, negotiated support

Every tier is described against *implemented* features, with anything else marked explicitly as roadmap. Onboarding and connector work is charged separately after estimating effort. A run’s maximum model and tool budget, document storage allowance, retention and overage behaviour are defined before a tier is offered. **Unlimited AI use is never promised.**

Cost model. Monthly service delivery cost is allocated compute + database + storage and backups + monitoring + model and embedding usage + support labour + identity/email costs, and gross margin is $(\text{revenue} - \text{service delivery cost}) / \text{revenue}$.

Per-run model cost cannot be quoted from this build

Token counts recorded by this repository come from the `fixture` provider, which computes them as `len(json.dumps(...)) // 4` — a cheap deterministic stand-in, **not a tokenizer and not provider usage**. They are usable as a *method* — measure `tokens_used` per run, multiply by the provider’s published per-token prices, add the embedding cost — and they are not usable as a cost figure. Any quoted per-run cost must come from a live-provider run with current published prices verified at the time of quoting.

Deployment options. Single-tenant SaaS hosted by the vendor, or customer-hosted Compose on the customer’s own VM for factories that will not let operational data leave the site.

Pilot plan. Measure time to identify a blocker, time spent preparing an order-status report, stale/missing-data rate, proposal acceptance rate and supervisor-rated usefulness — before and after. Progress from synthetic demo → read-only pilot → supervised actions → integrations, and only after validation at each step. **No productivity percentage is promised before before/after evidence exists.**

15 Limitations, and what has not been verified

15.1 Never executed in this environment

Table 17: Everything that is not verified, with the exact missing dependency.

Not verified	Why	Missing dependency
Live LLM provider run	no API key exists in this environment; the Anthropic adapter is exercised only against in-test stub SDK clients	LS_ANTHROPIC_API_KEY
Full evaluation (<code>make eval</code>)	the real <code>fastembed</code> run needs a ~100 MB model download plus a full embedding and query pass	user approval or CI
Load test (<code>make perf</code>)	sustained concurrent load for 60s is exactly what the machine heat policy forbids locally	user approval or CI
Full test suites (<code>make test</code> , <code>test-integration</code> , <code>test-all</code> , <code>security</code>)	same policy; only the files covering each change were run	user approval or CI
Any Docker operation	no Docker in this environment; images were never built or started	a Docker host or CI
Keycloak realm import and the hosted TLS login flow	depends on the above	a Docker host
Browser end-to-end flows and screenshots	Task 24 was dropped by the user as a scope decision	a reinstated E2E task
Real-factory validation of the formulas	synthetic data only; an IE/domain owner must validate production assumptions	a domain owner
Official report template	never supplied (§1.3)	the lecturer’s template
Lecturer-approved domain list	never supplied; the manufacturing domain is assumed, not confirmed	the approved list

15.2 Known functional limitations

- **No row-level security**; application-layer tenancy only (a documented pre-pilot gate).
- **No OCR**; scanned or image-only PDFs are rejected with a message rather than partially parsed. **No antivirus engine**; a structural marker scan only.
- **No approximate vector index**. Vector search is an exact scan — adequate at 30 documents, not at scale.
- **Single-instance rate limits**, in-process token buckets.
- **In-app notifications only**; no email or SMS.
- **No live ERP/MES connector**. The adapter interface (`fetch_changes`, `normalize`, `validate`, `apply_idempotently`, `checkpoint`) is documented and nothing is connected.
- **One style per order** (ADR-0007); no `order_items`.
- A note’s **unresolved** entity list and its classifier version at creation time are not persisted — there are no columns for them — so the list is only ever populated on the create response and the version shown is always the current process’s.
- The IE screen can only mark outliers on observations recorded in the current session, because no endpoint lists historical observations.
- **CANCELLED** exists in the `jobs` schema and is set by no code path.

15.3 Future work, in the order it should be done

1. Run the full verification set in CI and publish the artefacts: the full suites, `make eval` with `fastembed`, `make perf`, and the Docker build and Compose smoke flow.
2. Configure a provider key and record a live multi-agent run, then repeat the agent and fairness evaluation three times to report variance rather than a single number.
3. Implement database row-level security before onboarding a second organization.
4. Reinstate browser end-to-end coverage of the critical flows and capture the screenshots this submission lacks.
5. Add an approximate vector index and a shared rate-limit store before any multi-instance deployment.
6. Have an IE/domain owner validate the production assumptions behind the formulas before any real-factory use.

16 Conclusion

LineSense AI implements what the assignment asked for and, where it could not verify something, says so. Four agents really do interact — the planning agent revises its allocation because the RM agent’s number contradicted it, driven by a deterministic comparison rather than a model’s opinion. The protocol between them is versioned, schema-checked and server-verified against the run it claims to belong to. Retrieval is genuinely hybrid and genuinely access-controlled before content reaches a model. Approval integrity is enforced by row locks and version pinning rather than by convention, and is proved by tests that create real contention.

The part of the work that is easiest to undervalue is the part that refuses to claim things: the source label on every value, the abstention paths that return *unknown* instead of a plausible number, the degraded result that names what was lost, the evaluation directory that is empty rather than populated with a smoke run dressed up as a result, and the list in §15. A decision-support system for a factory is only worth deploying if the people using it can tell which of its statements are facts. That property was designed in from the first task, and it is the one this report most wants to be judged on.

References

Only sources actually used while building this system are listed.

1. pgvector — open-source vector similarity search for PostgreSQL. <https://github.com/pgvector/pgvector>
2. PostgreSQL 17 documentation, `SELECT (FOR UPDATE SKIP LOCKED)`. <https://www.postgresql.org/docs/current/sql-select.html>
3. PostgreSQL 17 documentation, row security policies. <https://www.postgresql.org/docs/17/ddl-rowsecurity.html>
4. OWASP Cheat Sheet Series — LLM Prompt Injection Prevention. https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html
5. OWASP Cheat Sheet Series — Session Management. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
6. FastAPI — deployment concepts. <https://fastapi.tiangolo.com/deployment/concepts/>
7. Vite — getting started guide. <https://vite.dev/guide/>
8. Playwright documentation (referenced by the plan for browser end-to-end testing; that task was dropped). <https://playwright.dev/docs/intro>
9. Keycloak — running Keycloak in a container. <https://www.keycloak.org/server/containers>

A Role permission matrix

Generated from `app.auth.policy.PERMISSIONS` by `scripts/gen_role_matrix.py`; regenerate rather than hand-editing. Columns: IE = `ie_engineer`, OA = `org_admin`, PL = `planner`, QM = `quality_manager`, SK = `storekeeper`, SV = `supervisor`, VW = `viewer`.

Permission	IE	OA	PL	QM	SK	SV	VW
admin:manage		x					
analysis:read	x	x	x	x	x	x	x
analysis:run			x			x	
audit:read		x				x	
capacity:read	x	x	x	x	x	x	x
document:read	x	x	x	x	x	x	x
document:upload	x	x		x		x	
ie:read	x	x	x	x	x	x	x
ie:write	x						
inventory:read	x	x	x	x	x	x	x
inventory:write					x		
note:create	x		x	x	x	x	
order:cancel						x	
order:create			x			x	
order:dispatch						x	
order:import			x			x	
order:read	x	x	x	x	x	x	x
order:transition			x			x	
quality:hold				x			
quality:inspect				x			
quality:read	x	x	x	x	x	x	x
quality:release				x			
recommendation:apply						x	
recommendation:decide						x	

B API surface

66 public paths / 70 operations, exported deterministically to `contracts/openapi.json` (OpenAPI 3.1.0, 151 component schemas) and used to generate the TypeScript client. Grouped:

Group	Representative endpoints
Identity	GET <code>/auth/login</code> , GET <code>/auth/callback</code> , POST <code>/auth/logout</code> , GET <code>/api/v1/me</code>
Health	GET <code>/api/health/live</code> , GET <code>/api/health/ready</code>
Orders	GET POST <code>/api/v1/factories/{f}/orders</code> , GET <code>/api/v1/orders/{id}</code> , transition and progress commands, GET <code>/api/v1/orders/{id}/history</code>
Imports	create batch, validate/preview, commit, row errors, template download
Analysis	POST <code>/api/v1/orders/{id}/analyses</code> , GET <code>/api/v1/runs/{id}</code> , GET <code>/api/v1/runs/{id}/events</code> , GET <code>/api/v1/orders/{id}/runs</code> , cancel, retry
Planning / capacity	GET <code>/api/v1/factories/{f}/lines</code> , GET <code>/api/v1/factories/{f}/capacity</code>
Inventory	material overview, ledger, receipts, lot acceptance, issues, corrections, reservations, release
IE	POST observations, mark outlier, operator aliases, GET <code>.../ie/{line}/{style}/analysis</code>
Quality	inspections, holds, releases, defect trends, shipment facts
Recommendations	scoped list, detail, POST <code>.../decision</code> , POST <code>.../apply</code>
Documents / search	upload, list, detail, version detail, download, GET <code>/factories/{f}/search</code> , GET <code>/citations/{chunk_id}</code>
NLP	POST GET <code>/api/v1/factories/{f}/notes</code> , GET <code>/api/v1/orders/{id}/status-summary</code>
Dashboard / admin / audit / notifications	factory dashboard, memberships, role grant/revoke, deactivate, policies, settings, scoped audit list, notifications list and mark-read
Internal (not in the public document)	POST <code>/internal/v1/agent-tasks</code> , GET <code>/internal/v1/agent-tasks/{id}</code>

Error bodies are uniform: `code`, a safe `message`, optional `field_errors`, `trace_id` and an optional server-controlled retry hint. 401/403 are used correctly, 404 avoids disclosing inaccessible resources, 409 covers stale and conflicting writes, 422 covers invalid input and 429 covers rate limits.

C Command reference

Target	What it does
<code>make bootstrap</code>	initialise the project-local PostgreSQL cluster, copy <code>.env</code> , <code>uv sync</code>
<code>make db-start /</code> <code>db-stop / db-init /</code> <code>db-reset-test</code>	manage <i>only</i> the project-local cluster on port 55432
<code>make migrate</code>	<code>alembic upgrade head</code>
<code>make migration-check</code>	<code>upgrade → downgrade → upgrade → alembic check</code> against the test database
<code>make seed</code>	deterministic synthetic dataset + the 30-document SOP corpus
<code>make idp</code>	the development-only OIDC provider (port 8090)
<code>make worker</code>	the durable worker (<code>python -m app.jobs</code>)
<code>make lint /</code> <code>typecheck / format</code>	<code>ruff + eslint / mypy + tsc / auto-format</code>
<code>make test /</code> <code>test-integration /</code> <code>test-all</code>	unit and web / real-PostgreSQL integration / both
<code>make contracts /</code> <code>contracts-check</code>	export OpenAPI + protocol schemas + TS client / detect drift
<code>make datasets-check</code>	validate the synthetic corpus and labelled datasets
<code>make docs-check</code>	every relative documentation link resolves
<code>make infra-check</code>	static validation of Compose, Caddyfile, realm and CI workflow
<code>make security</code>	secret scan + dependency audit + the <code>security</code> -marked suite
<code>make eval</code>	the full evaluation with the real embedder
<code>make perf</code>	the load generator (never run here)
<code>make backup /</code> <code>restore-check</code>	encrypted backup / timed backup-restore-verify exercise
<code>make build</code>	backend import check + production web build
<code>make web-install /</code> <code>web-dev / web-test /</code> <code>web-build</code>	frontend equivalents

There is no `make dev` target (each process is started explicitly — see `docs/user-guide.md` §4) and no `make test-e2e` target (browser end-to-end testing was dropped).

D Architecture decision records

ADR	Decision
0001	Modular monolith plus a separate worker; agents are components, not microservices
0002	PostgreSQL + pgvector as the single store; exact vector search first
0003	PostgreSQL job table with <code>FOR UPDATE SKIP LOCKED</code> , leases, fencing and at-least-once delivery
0004	OIDC + PKCE + opaque server sessions + CSRF double-check; a development-only OIDC provider locally, a Keycloak realm for Compose
0005	A custom versioned HTTP/JSON agent protocol — explicitly not A2A, not MCP
0006	One <code>LLMClient</code> boundary: Anthropic <code>claude-opus-5</code> with server-side refusal fallbacks, a deterministic <code>fixture</code> provider for CI, and a <code>disabled</code> degraded mode
0007	One style per order; a ledger plus a lockable <code>material_balances</code> row; embeddings on <code>chunks</code> ; one <code>approvals</code> row per recommendation
0008	A local environment without Docker or Java: a project-local PostgreSQL cluster on port 55432, and Compose/Keycloak validated statically only

Where to look next

`docs/user-guide.md` — setup and every screen and workflow. `docs/IMPLEMENTATION_STATUS.md` — the dated log of what was built and what was actually run. `docs/assessment/completion-matrix.md` — every requirement and acceptance gate with a verdict and evidence. `docs/assessment/` — commercialization, responsible AI, model and data cards, video script, contribution and AI-assistance logs, licences.